

CS5489 - Machine Learning

Lecture 11a - Large Language Models

Dept. of Computer Science, City University of Hong Kong

Outline

1. Tokenization
2. Language Models (LMs)
3. Large Language Models (LLMs)
4. LLMs: Problems and Mitigations
5. LLMs: Application and Future

```
In [1]: # setup
%matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100 # display larger images
import matplotlib

import numpy as np
import torch
import transformers
import nltk # for English tokenizer
import jieba # for Chinese tokenizer

# nltk.download('punkt_tab')
```

```
/Users/zzs/miniconda3/envs/cs5489/lib/python3.12/site-packages/tqdm/aut
o.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ip
ywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.
html
```

```
from .autonotebook import tqdm as notebook_tqdm
```

Tokenization

- How to represent natural language (text) inputs?
 - We need to first split the input into a sequence of **tokens**; this process is

called Tokenization.

- Why not simply split by spaces?
 - There are many languages that do not split words/tokens by spaces, such as Mandarin, Thai, etc.
 - Even in English, there is a growing number of new words (such as "LLM", "ChatGPT") and compounding words (such as "GPT-based"), which can explode the vocabulary.
- In modern NLP/LLM models, we split the input into a sequence of **subwords (tokens)**.
 - This is the unit that most APIs/Models count the LLM usage:
<https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them>

Tokenization Method 1: Rule-based

- Split by Spaces + Special Handling of Punctuations
- This used to be the default (and a reasonably good) method for English text processing (before the wide adoption of subword tokenization)
- NLTK has good support for this:

https://www.nltk.org/api/nltk.tokenize.word_tokenize.html

```
In [2]: sentence = "I'm studying machine learning and LLM-based methods in City University of Hong Kong."
words = nltk.tokenize.word_tokenize(sentence)
print(words)
```

```
['I', "'", 'm', 'studying', 'machine', 'learning', 'and', 'LLM-based', 'methods', 'in', 'City', 'University', 'of', 'Hong', 'Kong', '.']
```

Tokenization Method 2: Dictionary/Model-based

- For languages that there are no easy rules to split the sentence, we may treat it as a specific NLP task and use a dictionary or even learn a model to do this.
- There is a traditional NLP task for Chinese called Chinese Word Segmentation: <http://sighan.cs.uchicago.edu/bakeoff2005/>
- "jieba" is a widely used segmentation library for Chinese:

<https://github.com/fxsjy/jieba>

```
In [3]: sentence = "我在香港城市大学学习机器学习以及大模型。"
words0 = nltk.tokenize.word_tokenize(sentence) # this does not work!
print(words0)
words = jieba.cut(sentence)
print(list(words))
```

```
Building prefix dict from the default dictionary ...
Loading model from cache /var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/jieba.cache
['我在香港城市大学学习机器学习以及大模型。']

Loading model cost 0.265 seconds.
Prefix dict has been built successfully.
['我', '在', '香港城市大学', '学习', '机器', '学习', '以及', '大', '模型', '。']
```

Tokenization Method 3: Statistics-based

- Currently, (almost all) LLMs use statistics-based tokenization approaches for tokenization.
- Byte-pair encoding (BPE) is one of the mostly widely utilized approach: https://en.wikipedia.org/wiki/Byte-pair_encoding
- Sennrich et al. (2016) introduce the idea for neural machine translation and it is later widely utilized in NLP: <https://aclanthology.org/P16-1162.pdf>
- The idea is very simple: repeating the process of merging most frequently appearing nearby subtokens.

```
In [4]: tokenizer = transformers.AutoTokenizer.from_pretrained("Qwen/Qwen3-0.6
sentence_en = "I'm studying machine learning and LLM-based methods in
sentence_zh = "我在香港城市大学学习机器学习以及大模型。"
tokens_en = tokenizer.tokenize(sentence_en)
tokens_zh = tokenizer.tokenize(sentence_zh) # the subword representat
print([tokenizer.convert_tokens_to_string([z]) for z in tokens_en])
print([tokenizer.convert_tokens_to_string([z]) for z in tokens_zh])

['I', "'m", ' studying', ' machine', ' learning', ' and', ' L', 'LM', '
-based', ' methods', ' in', ' City', ' University', ' of', ' Hong', ' K
ong', ' .']
['我在', '香港', '城市', '大学', '学习', '机器', '学习', '以及', '大', '模
型', '。']
```

Language Models (LMs)

- After tokenization, we have our inputs: a sequence of tokens
 $T = [t_0, t_1, \dots, t_{N-1}]$, the task of LM is to build a probabilistic model $p(T)$ for the whole sequence.
- First, we need some good representations of the tokens so that they can be easily processed by the machine.
- The bag-of-words representation is not suitable here since we care about the full sequence and each token's position (it is an ordered sequence, not a bag).

- Decomposition:

$$p(T) = p(t_0) * p(t_1|t_0) * p(t_2|t_0, t_1) * \dots * p(t_{N-1}|t_0, t_1, \dots, t_{N-2})$$

- Therefore, the main work is to model the **Next Token Prediction**:

$$p(t_i|t_0, \dots, t_{i-1})$$

LM Method 1: N-gram LM

- LM can be as simple as counting, but we need some **independence assumptions** here.
- The N-gram LM assume that $p(t_i|t_0, \dots, t_{i-1}) \approx p(t_i|t_{i-n}, \dots, t_{i-1})$.
- Unigram: $p(t_i)$, Bi-gram: $p(t_i|t_{i-1})$, Tri-gram: $p(t_i|t_{i-2}, t_{i-1})$, ...
- For LM evaluation, we usually calculate perplexity (PPL):

<https://en.wikipedia.org/wiki/Perplexity>

- Take 2 as the base: $PPL = 2^{-\frac{1}{N} \sum_i \log_2 p(t_i)}$

```
In [5]: # let's build some simple n-gram LM
from collections import Counter, defaultdict
corpus = [
    ["I", "am", "a", "student"],
    ["I", "study", "LLM"],
    ["LLM",

# count everything
unigram, bigram = Counter(), defaultdict(Counter)
for seq in corpus:
    prev = "<BOS>" # special beginning token
    for tok in seq + ["<EOS>"]: # add special ending token
        unigram[tok] += 1
        bigram[prev][tok] += 1
        prev = tok

# normalization
prob_unigram = {k: v/sum(unigram.values()) for k,v in unigram.items()}
prob_bigram = {p: {k: v/sum(d.values()) for k,v in d.items()} for p, d

print(prob_unigram)
print(prob_bigram)
```

```
{'I': 0.15384615384615385, 'am': 0.07692307692307693, 'a': 0.0769230769
2307693, 'student': 0.07692307692307693, '<EOS>': 0.23076923076923078,
'study': 0.07692307692307693, 'LLM': 0.15384615384615385, 'is': 0.07692
307692307693, 'cool': 0.07692307692307693}
{'<BOS>': {'I': 0.6666666666666666, 'LLM': 0.3333333333333333}, 'I': {'
am': 0.5, 'study': 0.5}, 'am': {'a': 1.0}, 'a': {'student': 1.0}, 'stud
ent': {'<EOS>': 1.0}, 'study': {'LLM': 1.0}, 'LLM': {'<EOS>': 0.5, 'i
s': 0.5}, 'is': {'cool': 1.0}, 'cool': {'<EOS>': 1.0}}
```

```
In [6]: # calculate the prob of a seq
def get_logprobs(s):
    logprobs = []
    prev = "<BOS>" # special beginning token
```



```

for tok in s + ["<EOS>"]:
    prob = prob_bigram.get(prev, {}).get(tok, 0.)
    logprobs.append(np.log2(prob).item())
    prev = tok
return logprobs

for seq in (["I", "am", "a", "student"], ["student", "is", "cool"]):
    _logprobs = get_logprobs(seq)
    _nll = - np.mean(_logprobs)
    _ppl = np.exp2(_nll)
    print(f"Sequence={seq}, Logprobs={_logprobs}, NLL={_nll}, PPL={_ppl}")

```

```

Sequence=['I', 'am', 'a', 'student'], Logprobs=[-0.5849625007211563, -1.0, 0.0, 0.0, 0.0], NLL=0.31699250014423125, PPL=1.2457309396155174
Sequence=['student', 'is', 'cool'], Logprobs=[-inf, -inf, 0.0, 0.0], NLL=inf, PPL=inf

```

```

/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_23235/1567174493.py:7: RuntimeWarning: divide by zero encountered in log2
  logprobs.append(np.log2(prob).item())

```

What are the main problems of n-gram LMs?

- Sparsity problem: The estimations of low-frequency words or n-grams are poor because of lack of data, for OOV words and phrases, the probabilities will be zero and we need smoothing: <https://aclanthology.org/P96-1041.pdf>
- Representation problem: we cannot well represent the semantic meaning of the tokens, and we need (neural) distributed representations
- Limited-context problem: we have to make strong assumptions that words only depend on nearby tokens, and this assumption is not true in many times; we need stronger models that can take more context into considerations (such as self-attention based models)

LM Method 2: (Early) Neural LM

- The idea is to adopt the distributed representations for each token to allow better modeling of the semantics.
- The core innovation is the **word embedding**, which utilizes a vector to represent each word/token.
- Two ways to understand it: 1) a dictionary from a word index to a word vector, 2) sparse matrix multiplication with one-hot input

```

In [7]: # create some random init word embeddings
dictionary = {k: i for i, k in enumerate(prob_unigram.keys())}
print(dictionary)
word_embeddings = np.random.random([len(dictionary), 100]) # [vocab_s

```

```

# map to a list of embeddings
seq = ["I", "am", "a", "student", "<EOS>"]
arr_emb = np.stack([word_embeddings[dictionary[t]] for t in seq], 0)
print(arr_emb.shape)

# matrix multiplication
arr_input = np.zeros([len(seq), len(dictionary)]) # [seq_length, voca
for i, s in enumerate(seq):
    arr_input[i][dictionary[s]] = 1. # one-hot
arr_emb2 = np.matmul(arr_input, word_embeddings)
print(arr_emb2.shape)

```

```

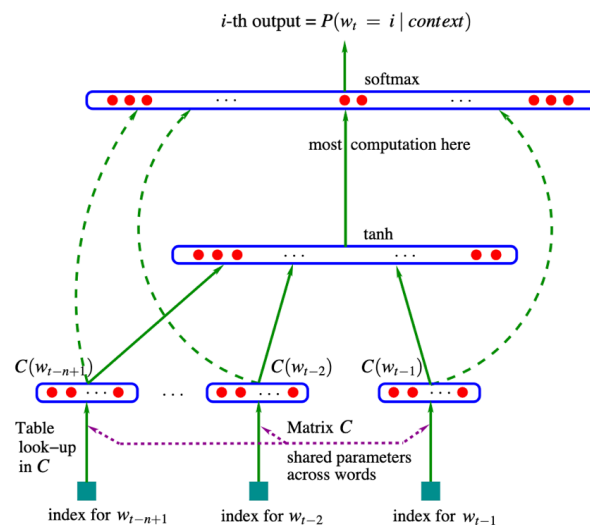
{'I': 0, 'am': 1, 'a': 2, 'student': 3, '<EOS>': 4, 'study': 5, 'LLM': 6, 'is': 7, 'cool': 8}
(5, 100)
(5, 100)

```

After the embedding layer, we map the original text input to a hidden-vector styled "neural" representations, and we can then use our favourite neural models to deal with them. An MLP can be used together with the local window constraint to make a neural N-gram LM. The output is a softmax over all the tokens in the vocab for the next token prediction.

One of the earliest neural LM:

<https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf>

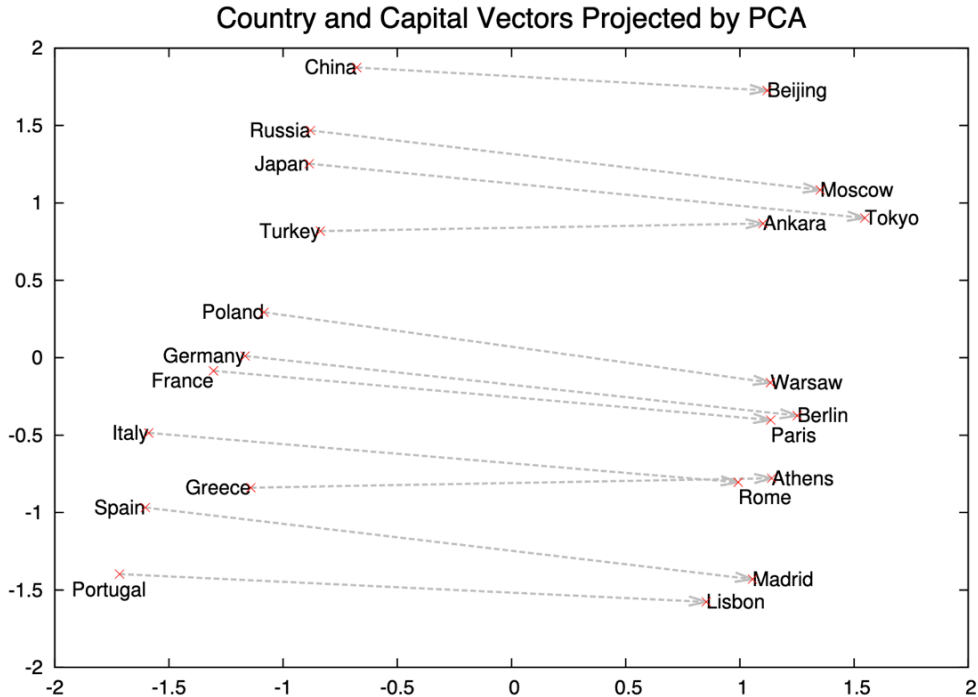
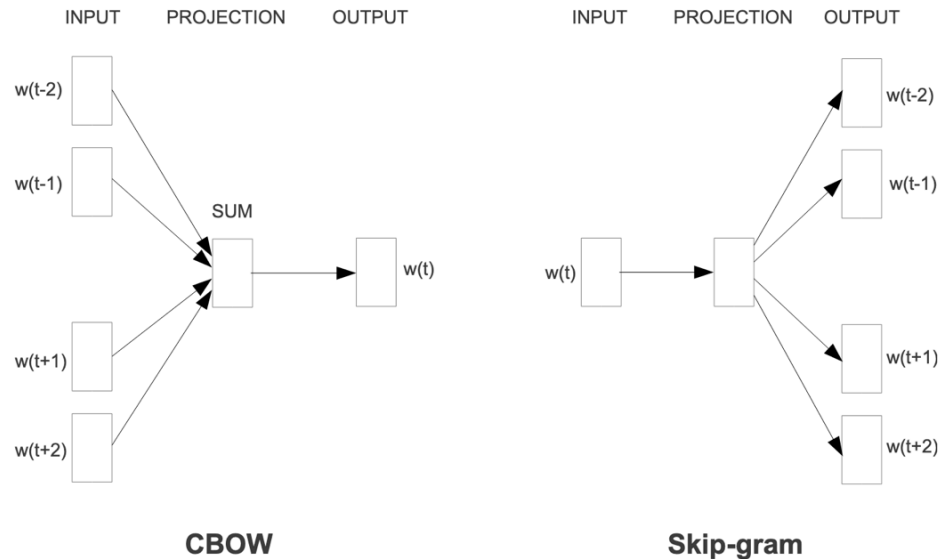


There is one more problem, how to obtain the word embeddings. Of course, you can random initialize them and train the full the LM together with other parameters in the NN, which is basically what we have done previously. Another way is that you can pre-train word vectors with simpler objectives:

- Word2Vec: <https://arxiv.org/pdf/1301.3781>, <https://arxiv.org/pdf/1310.4546>
- Glove: <https://nlp.stanford.edu/projects/glove/>

Let's take a look at the famous word2vec (special neural LM without hidden layers) and the interesting properties of the learned word vectors:

- We do not actually need a LM to predict the next token, so we can freely predict tokens based on the surrounding tokens (not just previous ones)
- There are interesting geometric properties of the learned word vectors: "king - man + woman $\approx$ queen"



Still the major problem of limited context window cannot be resolved, especially when there are long-range dependencies:

- "The cake that I bought yersterday is"? vs "The book that I bought yersterday"

is" (choices = ["delicious", "interesting"])

- Here, we need a 6-gram model to differentiate between these two, which is not quite flexible for a static N-gram neural model to deal
- Maybe we need better models?

LM Method 3: Transformer LM

- To be noted, there have been many other models for better LMs, including RNNs (recurrent neural networks), CNNs, etc.
- But finally, most modern LMs are built upon the Transformer model (attention is all you need): <https://papers.neurips.cc/paper/7181-attention-is-all-you-need.pdf>
- We have learned the Transformer architecture in previous lectures, let's review its core attention mechanism:

```
In [8]: # the simplist version of attention
def attention(query, key, value):
    scale_factor = 1 / (query.size(-1) ** 0.5)
    attn_weight = query @ key.transpose(-2, -1) * scale_factor
    attn_prob = torch.softmax(attn_weight, dim=-1)
    attn_res = attn_prob @ value
    return attn_res

# check with torch's sdpa
t_input = torch.rand([1, 2, 5, 10]) # [batch_size, head_number, seq_l
res1 = attention(t_input, t_input, t_input)
res2 = torch.nn.functional.scaled_dot_product_attention(t_input, t_inp
print(torch.allclose(res1, res2))
```

True

- The benefit of self-attention is that it does not need to assume a fixed-size of window of inputs, and it can take any number of inputs and compute a weighted sum of the value vectors for them. In this way, we do not need any independence assumption if using self-attention for LM (of course you can still add some assumptions by masking out long-range tokens).
- There is one more thing to deal with for LM: causal mask, the prediction of the current token CANNOT depend on the future tokens. This can be achieved by adding mask to the attention calculation.

```
In [9]: # attention with causal mask
def attention(query, key, value, is_causal=False):
    L, S = query.size(-2), key.size(-2)
    scale_factor = 1 / (query.size(-1) ** 0.5)
    attn_weight = query @ key.transpose(-2, -1) * scale_factor
```

```

if is_causal:
    attn_bias = torch.zeros(L, S, dtype=query.dtype, device=query.device)
    temp_mask = torch.ones(L, S, dtype=torch.bool).tril(diagonal=0)
    attn_bias.masked_fill(temp_mask.logical_not(), float("-inf"))
    attn_weight = attn_weight + attn_bias
    attn_prob = torch.softmax(attn_weight, dim=-1)
    attn_res = attn_prob @ value
    return attn_res

# check with torch's sdpa
t_input = torch.rand([1, 2, 5, 10]) # [batch_size, head_number, seq_l
res1 = attention(t_input, t_input, t_input, is_causal=True)
res2 = torch.nn.functional.scaled_dot_product_attention(t_input, t_inp
print(torch.allclose(res1, res2))

```

True

The other components of Transformer-based LMs are the same as other neural models, such as the input embeddings, output softmax, the internal MLPs, etc. Another very good property of Transformer is that it is easy to parallelize and can scale, which ultimately leads to LARGE scaled trained LLMs.

Large Language Models (LLMs)

- As the name suggests, they are just LMs that are large! Extraordinarily large! (Nothing too magical ...)
- (But of course, there have been many updates on the Transformer architecture: such as activation functions, RMSNorm, Rotary position encoding, grouped-query attention, etc)
- There is the **scaling law**: <https://arxiv.org/pdf/2001.08361>
- They are still LMs (though they are super large), then why they become the game changers?

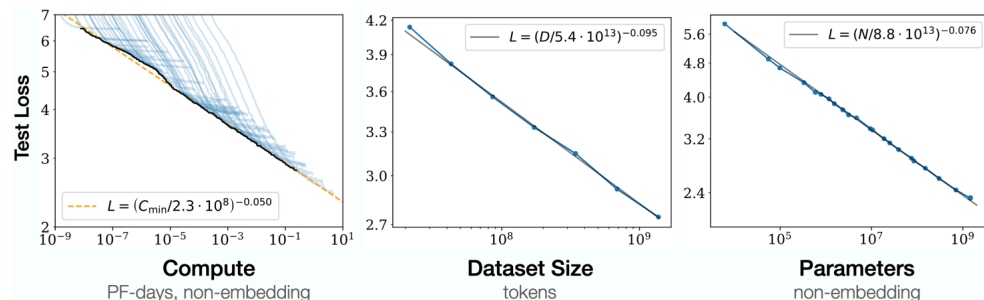
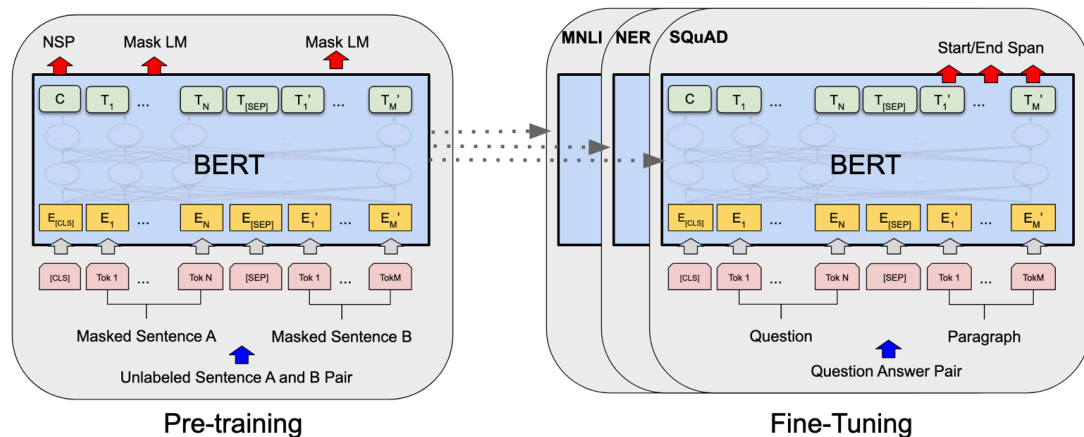


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Turning Point 0: ELmo and BERT (2018) w/ Large-

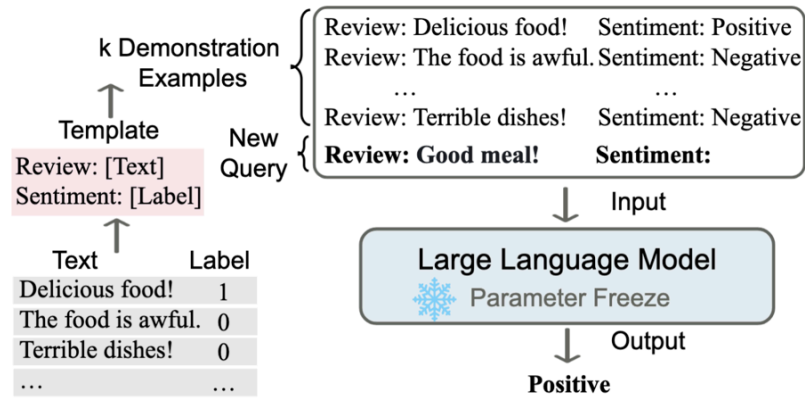
scale Pre-training

- Deep bi-directional (LSTM-based) LM pre-training:
<https://arxiv.org/pdf/1802.05365>
- Deep bi-directional masked LM pre-training with Transformer:
<https://arxiv.org/pdf/1810.04805>
- Generative Pre-trained Transformer 1 (GPT-1):
<https://en.wikipedia.org/wiki/GPT-1>
- The "pre-training then fine-tuning paradigm":



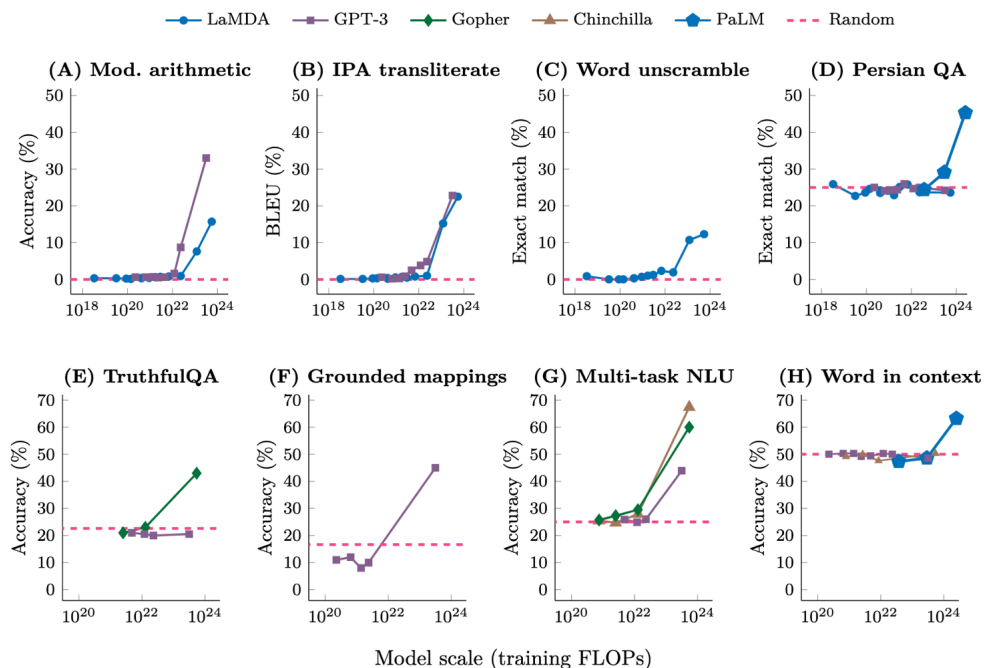
Turning Point 1: GPT-3 (2020) w/ In-Context Learning

- GPT-3: Language Models are Few-Shot Learners
(<https://arxiv.org/pdf/2005.14165>)
- In-Context Learning w/ Few-shot prompting means that you do NOT need to tune the model's parameters, just provide them with some similar examples in the input context, and they will solve the problems in a similar way. (The mechanism and knowledge for this is already learned in the LM pre-training stage.)
- See this survey for more details (if you are interested):
<https://arxiv.org/pdf/2301.00234>



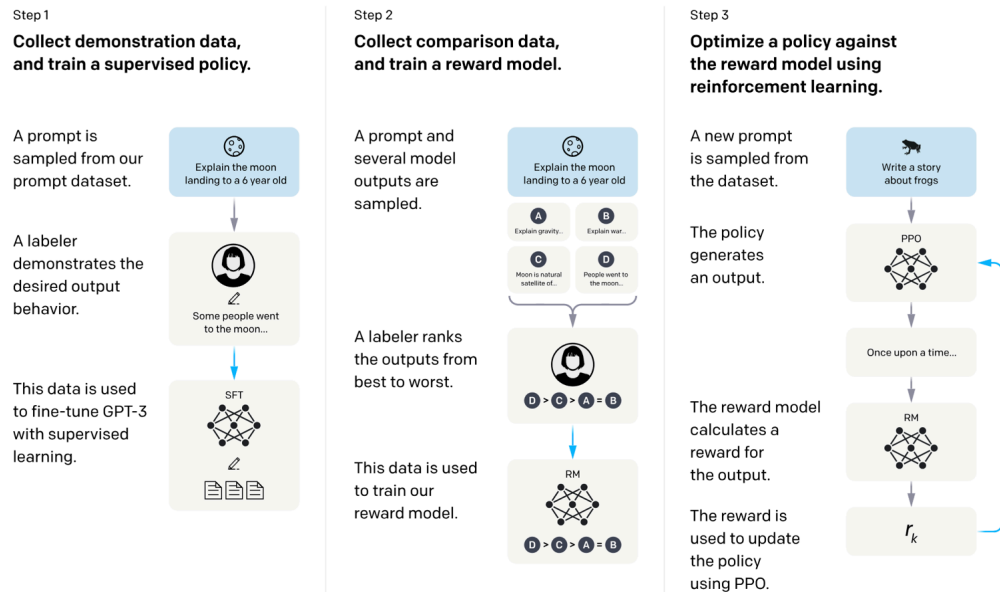
- Emergent Abilities of Large Language Models:

<https://arxiv.org/pdf/2206.07682>



Turning Point 2: InstructGPT (2022) w/ RLHF

- Training language models to follow instructions with human feedback: <https://arxiv.org/pdf/2203.02155>
- A plain LM is less useful; but an LM that can respond to human's queries and correspondingly answering is very useful!
- How to obtain such LMs? (We do not have good evaluation metrics for these open-ended problems) -> Use a trained reward model and align things to human preference.
- Three stages of training: 1) SFT, 2) Reward Model, 3) RLHF



SFT vs RL

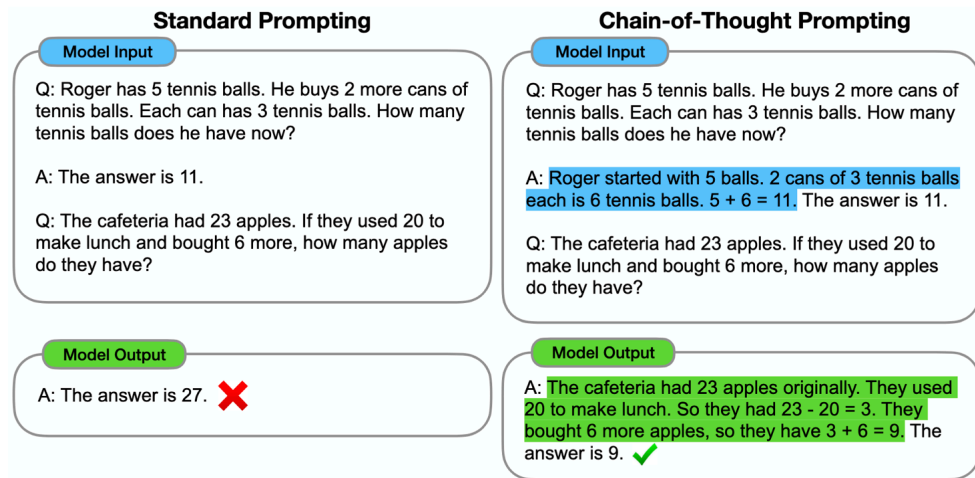
- In SFT (supervised fine-tuning), we are still tuning the LM with the cross-entropy loss, similar to pre-training. We have some fixed dataset with pairs of (prompt, answer), and then tune the models to maximize the predictions of the answering tokens.
- In RL (reinforcement learning), our target is to maximize a specific reward that the LM finally obtain after generating an answer.

LMs can be trained to follow instructions!

- This means that we can basically turn many of our problems in such query-answer format and directly ask LMs to solve them!
- The remaining part will be just prompt engineering??
- This finally leads to ChatGPT (2022) and GPT4 (2023) (also including another important aspect of code-based training).

Turning Point 3: OpenAI-o1 (2024) and Deepseek-R1 (2025) w/ Reasoning

- The idea originates from chain-of-thought (CoT): <https://arxiv.org/pdf/2201.11903>
- We do not just need LLMs to act as chatbots to chat with us, we need them to solve more complex tasks, which requires "thinking".
- A series of intermediate reasoning steps can significantly improve the ability of LLMs to perform complex reasoning.



- The main problem is how to obtain such intermediate thinking rationales with high quality?
- RLVR (RL with Verifiable Rewards) is enough: <https://arxiv.org/pdf/2501.12948>
- Recent LLMs usually have the "think" mode, where will allow the model to perform deep thinking before answering the question.

```
In [10]: def my_generate(_messages, _model, _tokenizer, _thinking):
    text = _tokenizer.apply_chat_template(
        _messages,
        tokenize=False,
        add_generation_prompt=True,
        enable_thinking=_thinking # Switches between thinking and non-
    )
    model_inputs = _tokenizer([text], return_tensors="pt").to(_model.device)
    # conduct text completion
    generated_ids = _model.generate(
        **model_inputs,
        max_new_tokens=512, # you can set to a larger value to see more
    )
    output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()
    output_str = _tokenizer.decode(output_ids, skip_special_tokens=True)
    return output_str

# let's try a small model
model_name = "Qwen/Qwen3-0.6B"
model = transformers.AutoModelForCausalLM.from_pretrained(model_name,
    prompt = "Give me a short introduction to deepseek R1."
    messages = [{"role": "user", "content": prompt}]

for thinking in [True, False]:
    output_str = my_generate(messages, model, tokenizer, thinking)
    print(f"Think={thinking}, reply={output_str}\n")
```

``torch_dtype` is deprecated! Use `dtype` instead!`

Think=True, reply=<think>

Okay, the user wants a short introduction to DeepSeek R1. First, I need to make sure I understand what DeepSeek R1 is. From what I know, DeepSeek is a deep learning model developed by Alibaba Group. They have released R1 in recent years.

I should start by mentioning that DeepSeek R1 is a high-performance model. It's built on Alibaba's large-scale data and AI infrastructure. Then, I can highlight its key features like support for multiple languages, efficient training, and real-time processing. Also, since it's a model, it's used in various applications like language translation, content creation, and other AI-driven tasks.

I need to keep it concise but informative. Avoid technical jargon if possible, but still convey the importance. Make sure the introduction flows well and includes both the model's capabilities and its applications. Double-check that all key points are covered without being too detailed.

</think>

DeepSeek R1 is a high-performance deep learning model developed by Alibaba Group, designed for efficient language processing and AI-driven tasks. It leverages advanced training techniques and large-scale data to deliver rapid performance, supporting multiple languages and enabling real-time applications.

Think=False, reply=DeepSeek R1 is a high-performance, AI-driven video and audio processing platform designed for creators and developers. It offers advanced features like real-time editing, intelligent content creation, and seamless integration with various devices and platforms.

LLM Training

- Pre-training: training on vast amount of (unlabeled) web data to gain basic knowledge; the LLM can already do simple in-context learning after this stage.
- Post-training: SFT/RLHF/RLVR/... to allow the LLM to align to human preference, being able to follow instructions and solve more complex problems.

LLM Inference

- Inference: Simply predict the next token one at a time (usually need sampling rather than greedy search)
- There are interesting LLM-System problems to allow efficient LLM inference serving: check vllm (<https://github.com/vllm-project/vllm>) or sglang (<https://github.com/sgl-project/sglang>) to see more related techniques.

```
In [11]: # what happens underlying model.generate
def simple_generate(m, t, t_input, max_token: int):
    # encoding
    assert len(t_input) == 1, "For simplicity, we only handle one inst
    _output = m(t_input)
    ret = []
    # sample one token at one time
    for _ in range(max_token):
        _cache = _output.past_key_values
        _logits = _output.logits
        # sampling the next token
        next_id = torch.multinomial(_logits[0, -1].softmax(-1), 1).ite
        ret.append(next_id)
        if next_id == t.eos_token_id:
            break # end for EOS
    # modeling
    _output = m(torch.as_tensor([[next_id]]).to(t_input), past_key
    return ret

# --
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
    enable_thinking=False # Switches between thinking and non-thinking
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)
output_ids = simple_generate(model, tokenizer, model_inputs['input_ids
output_str = tokenizer.decode(output_ids)
print(output_str)
```

DeepSeek R1 is a mobile device app that provides a comprehensive and intuitive text-to-speech service for users to convert text into speech. It's designed for quick and clear communication, making it ideal for daily tasks and personal use.<|im_end|>

LLMs: Problems and Mitigations

LLMs still have many limitations and problems, including hallucination, grounding problems, efficiency issues, lack of interpretability, safety issues, bias and ethics issues, ...

Hallucination & RAG

- LLMs may generate false, fabricated, or inaccurate information that is presented as fact. This occurs because LLMs are designed to predict the next most plausible word based on their training data, and they can fill in gaps with incorrect information, even when they are unsure of the correct

answer.

- In ordinary LLMs, there are NO explicit memory or knowledge modules, all the knowledge is stored inside the parameters.
- How to deal with long-tail facts and new knowledge?

```
In [13]: # here we are guiding the LM to produce false information, but there are
messages = [{"role": "user", "content": "Give me the detailed process"}]
output_str = my_generate(messages, model, tokenizer, False)
print(output_str)
```

China did not win the 2008 FIFA World Cup. This is a factual and historical fact. The 2008 World Cup was won by **Brazil**, not China. The event took place in **Porto Alegre, Brazil**, and was held from **July 12 to July 15, 2008**.

Retrieval-augmentation generation (RAG) is a technology to augment the context of LLMs with related information from external knowledge base. It can be one of the methods to mitigate hallucination problem of LLMs by providing extra information.

```
In [18]: related_information = ["FIFA world cup was held in the following years"]
messages = [{"role": "user", "content": f"Here is related information: {related_information}"}]
output_str = my_generate(messages, model, tokenizer, False)
print(output_str)
```

China did not win the **2008 FIFA World Cup**. The **2006 FIFA World Cup** was won by **Italy**, the **2010 World Cup** by **Spain**, the **2014 World Cup** by **Germany**, the **2018 World Cup** by **France**, and the **2022 World Cup** by **Argentina**.

If you're looking for information about **China's participation or achievements in other World Cups**, let me know!

Grounding & Agent

- LLMs themselves can only chat, but cannot do anything further, especially when we need grounding in a specific environment.
- But being used as the central control engine and augmented with perception and action modules, it can work as an agent to solve much more real world tasks.
- Some introductions: <https://lilianweng.github.io/posts/2023-06-23-agent/>

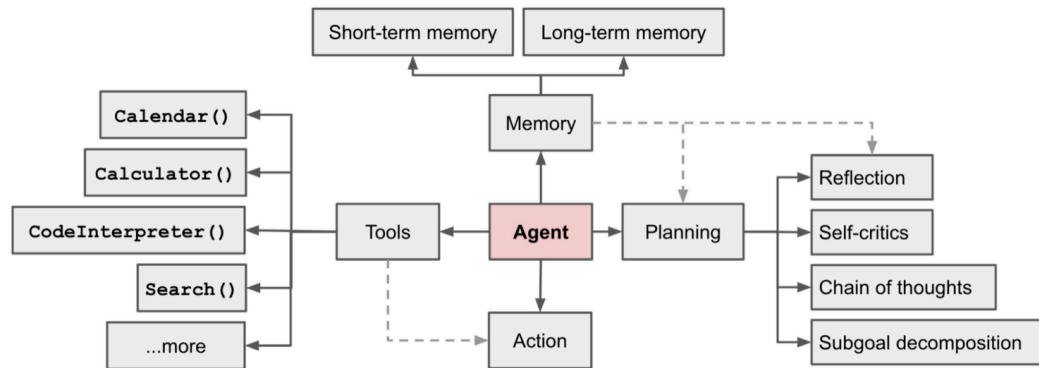


Figure 1: Overview of a LLM-powered autonomous agent system.

```
In [23]: input_inst = """You are a helpful agent tasked to perform a web-browsi
You will receive a simplified representation of the current web page's

The valid actions include:
- click [ID]: click a specific link, the [ID] should be the index of t
- type [ID] [content]: type "content" into the input box of [ID]
- goto [URL]: goto a specific URL
- finish [answer]: finish the task with the [answer]
""" # the main instruction

input_obs = """Here is the current observation:
[1] [Text] Google's Homepage
[2] [InputBox] Input your search query
[3] [Button] Search
[4] [Link] advertisement
[5] [Link] Wikipedia: 2022 FIFA World Cup, Argentina were crowned the
[6] [Link] FIFA's homepage: ...
""" # the current observation

input_hist = """Here is the history steps that you have finished:
Step 1: Go to Google's Homepage
Step 2: Search with the query "2022 FIFA world cup winner"
""" # history steps

input_prompt = """The target task is to answer the following query:
Who is the winner of 2022 FIFA world cup?

Please generate your action according to the action specifications.
"""

messages = [{"role": "user", "content": "\n".join([input_inst, input_o
output_str = my_generate(messages, model, tokenizer, True)
print(output_str)
```

<think>

Okay, let's see. The user wants to know who the winner of the 2022 FIFA World Cup was. The current web page shows a few links. The first link is "Google's Homepage", which we already visited. Then there's an input box where they can type a query, and a button to search. Then there are two links: one is "advertisement" and another is "Wikipedia: 2022 FIFA World Cup, Argentina were crowned...".

The history steps so far are step 1: Go to Google's Homepage, step 2: Search with query "2022 FIFA world cup winner". Now, the task is to answer the question about the winner.

Looking at the current observation, the link provided is "Wikipedia: 2022 FIFA World Cup, Argentina were crowned...". So the answer should be Argentina. Since the user's query is asking for the winner, and the link points directly to that information, the correct action would be to click on that link. But wait, the valid actions include clicking on the ID, which in the current observation is [ID]. Let me check the structure again.

The links are labeled [4] [Link] advertisement and [5] [Link] Wikipedia: 2022 FIFA World Cup, Argentina were crowned... So the ID for the Wikipedia link is 5. So the correct action would be to click on the ID 5. That would take the user to the Wikipedia page, where the answer is provided. Therefore, the next action is to click on the link with ID 5.

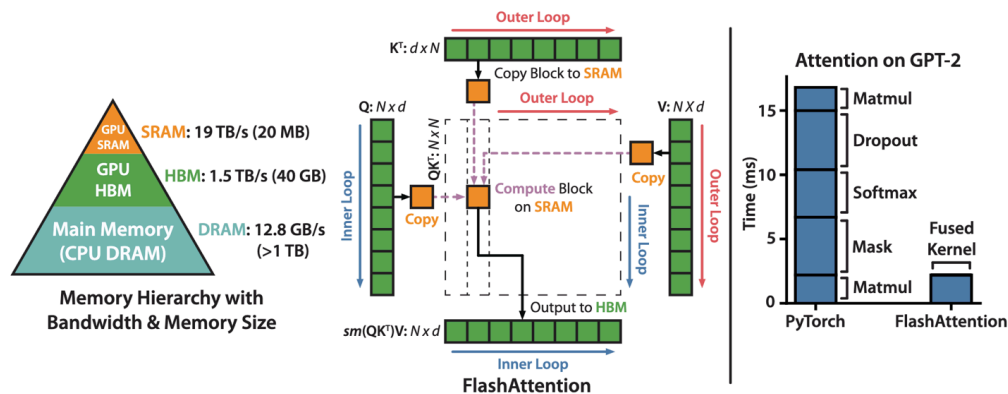
</think>

click [5]

(note, sometimes, with the small 0.56B model, it still cannot handle the task well. Larger models will get better performance.)

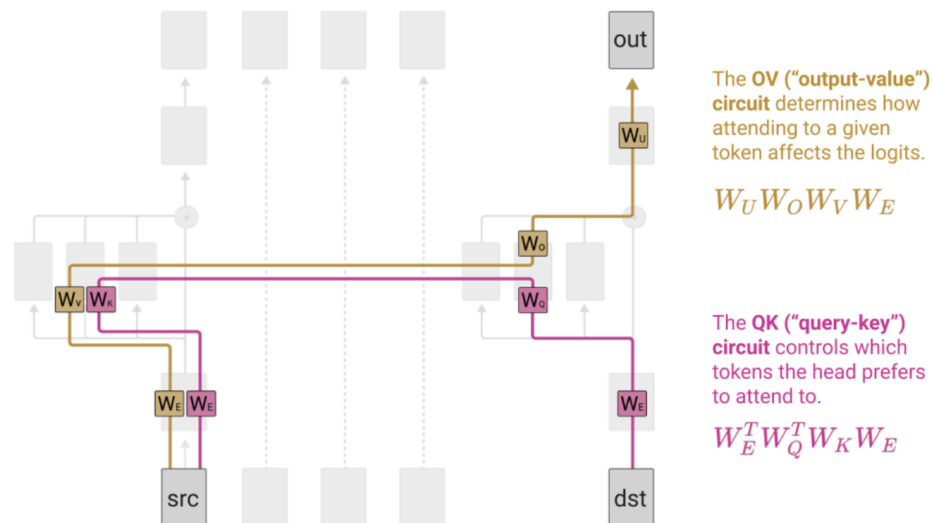
Efficiency

- LLMs are so large and costly, it will be difficult to train and deploy them.
- Many techniques have been investigated: quantization, compression, distillation, efficient attention.
- This is not only an ML algorithm problem, this is also an ML system problem, where we need model-hardware co-designs.
- A good example is flash attention: <https://arxiv.org/pdf/2205.14135>
<https://github.com/Dao-AILab/flash-attention>



Interpretability and Safety

- NNs are black-boxes, LLMs are large black-boxes; we still do not fully understand their internal mechanisms.
- This is important for the safety of using them; how to make sure they can reliably generate reasonable outputs and do not go out of control?
- Transformer Circuits Thread: <https://transformer-circuits.pub/>



Bias and Ethics Issues

- Bias Issues: Stereotypes, Representational Bias, Algorithmic Discrimination, Toxic and Harmful Content
- Ethics Issues: Privacy Concerns, Intellectual Property and Copyright, Job Displacement, Environmental Impact
- We need to carefully think more ...

LLMs: Applications and Future

- Applications:
 - Chatbot
 - agent systems for QA, web, GUI, SWE, even for robots etc
 - personal assistant
 - ...
- Main research questions:
 - Is Transformer the "ultimate" model architecture for LMs?
 - Is next-token-prediction the "ultimate" training method for LMs?
 - How to better incorporate more modalities (image, audio, video, etc) together with the LMs?

In []:

CS5489 - Machine Learning

Lecture 11b - Vision Language Models (VLM)

Dept. of Computer Science, City University of Hong Kong

Outline

1. Tokenization
2. Visual Application
3. Vision Language Models
4. Multi-Modal Large Language Models (MLLM)
5. VLM: Prolems and Mitigations

```
In [33]: # setup
%matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100 # display larger images
import matplotlib

import numpy as np
import torch
from torch import nn as nn
import torch.nn.functional as F
from torch.utils.checkpoint import checkpoint
from torchvision.transforms import Normalize, Compose, ToTensor, Resize
import transformers
from transformers import pipeline, AutoProcessor, AutoModelForQuestion
import nltk # for English tokenizer

from PIL import Image
import requests

from collections import OrderedDict
from itertools import repeat
import collections.abc
from dataclasses import dataclass
from diffusers import AutoPipelineForText2Image
import math
```

```
from typing import Tuple, Union, Callable, Optional
```

Tokenization

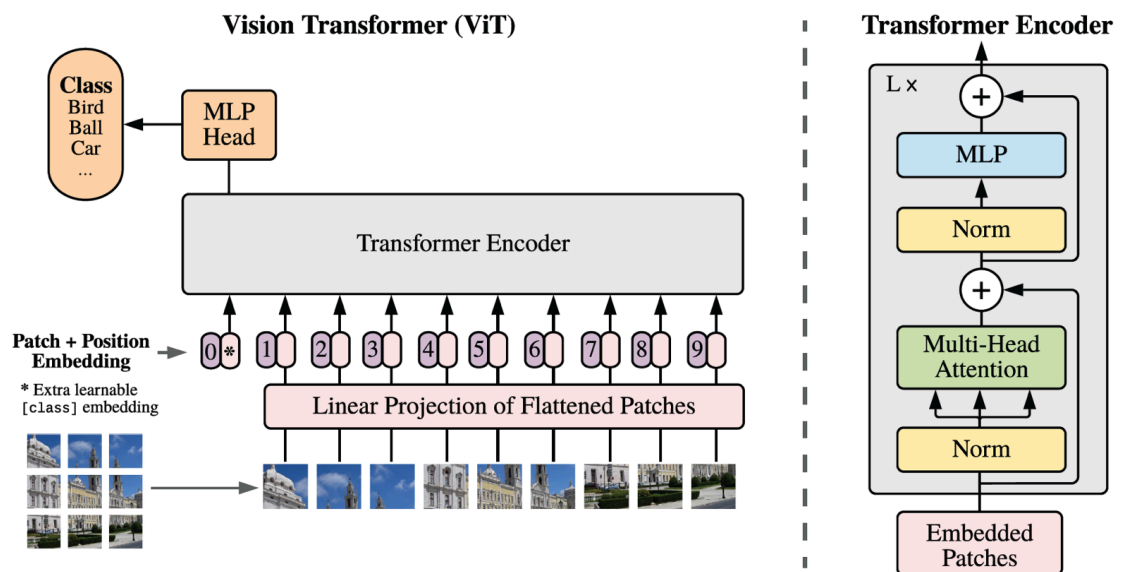
Image Input: Raw image (e.g., 224×224×3 pixels)

Patch Creation:

- Divide image into fixed-size patches (e.g., 16×16 pixels)
- Example: 224×224 image → 196 patches (14×14 grid of 16×16 patches)

Patch Embedding:

- Flatten each patch into 1D vectors
- Linear projection to embedding dimension
- Add positional embeddings to preserve spatial information



```
In [7]: mean = (0.48145466, 0.4578275, 0.40821073) # OpenAI dataset mean
std = (0.26862954, 0.26130258, 0.27577711) # OpenAI dataset std
image_size = 224

def _convert_to_rgb(image):
    return image.convert('RGB')

normalize = Normalize(mean=mean, std=std)

transforms = [
    Resize(image_size, interpolation=InterpolationMode.BICUBIC),
    CenterCrop(image_size),
    _convert_to_rgb,
    ToTensor(),
```

```

        normalize,
    ]
    transforms = Compose(transforms)

```

```

In [ ]: def _ntuple(n):
        def parse(x):
            if isinstance(x, collections.abc.Iterable):
                return x
            return tuple(repeat(x, n))
        return parse
    to_2tuple = _ntuple(2)

    patch_size = 16
    width = 768
    image_size = to_2tuple(image_size)
    patch_size = to_2tuple(patch_size)

    grid_size = (image_size[0] // patch_size[0], image_size[1] // patch_size[1])
    conv1 = nn.Conv2d(in_channels=3, out_channels=width, kernel_size=patch_size)

    scale = width ** -0.5
    class_embedding = nn.Parameter(scale * torch.randn(width))
    positional_embedding = nn.Parameter(scale * torch.randn(grid_size[0] * grid_size[1]))

    img_path = "imgs/testing.jpg" # Image of cats on a couch
    raw_image = Image.open(img_path).convert('RGB')

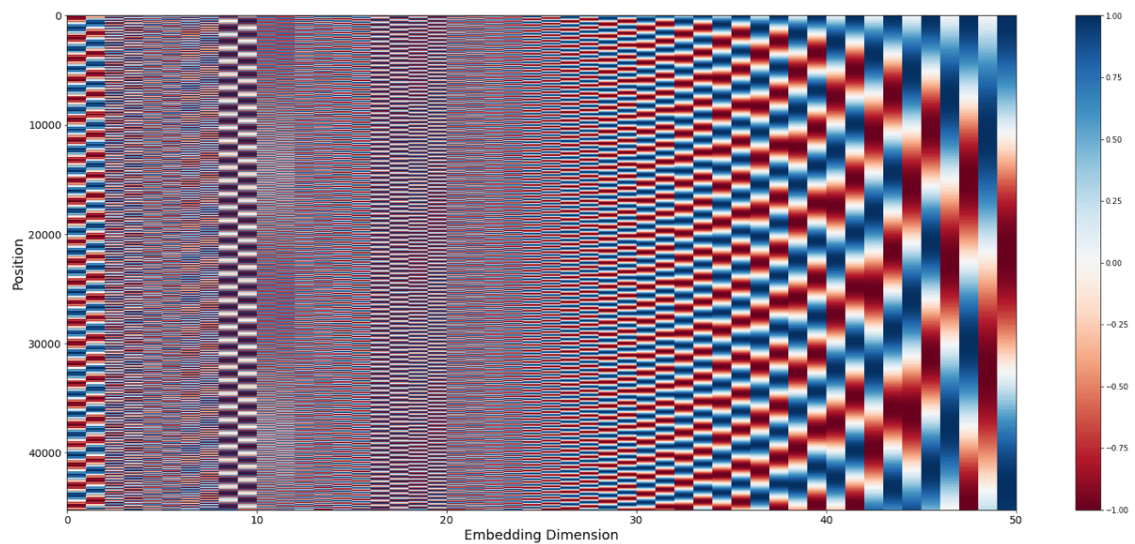
    x = transforms(raw_image).unsqueeze(0)
    x = conv1(x) # shape = [*, width, grid, grid]
    print('The convolution layer is implemented for image tokenization')
    x = x.reshape(x.shape[0], x.shape[1], -1) # shape = [*, width, grid * grid]
    print('There are {} grids splitted from the image'.format(x.shape[-1]))
    x = x.permute(0, 2, 1) # shape = [*, grid * grid, width]
    x = torch.cat(
        [class_embedding.to(x.dtype).view(1, 1, -1), x], dim=1) # shape = [1, 1, width + grid * grid]
    x = x + positional_embedding.to(x.dtype)

    print('The final size is', x.squeeze().detach().numpy().shape, 'after')

```

Absolute Position Embedding

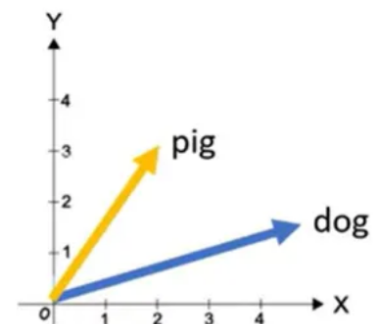
- Learnable (shown in code)
- Absolute embedding (Sinusoidal Functions of different frequencies)
- Relative Positional Embeddings



Relative Positional Embeddings

- Apply a rotation to the word vector (angle of rotation (θ) is proportional to the word's position in the sentence) [Reference](#)
- *Stability of Vectors*: Adding tokens at the end of a sentence doesn't affect the vectors for words at the beginning, facilitating efficient caching.
- *Preservation of Relative Positions*: If two words, say "pig" and "dog," maintain the same relative distance in different contexts, their vectors are rotated by the same amount. Consequently, the dot product between these vectors, remains constant

The pig chased the dog



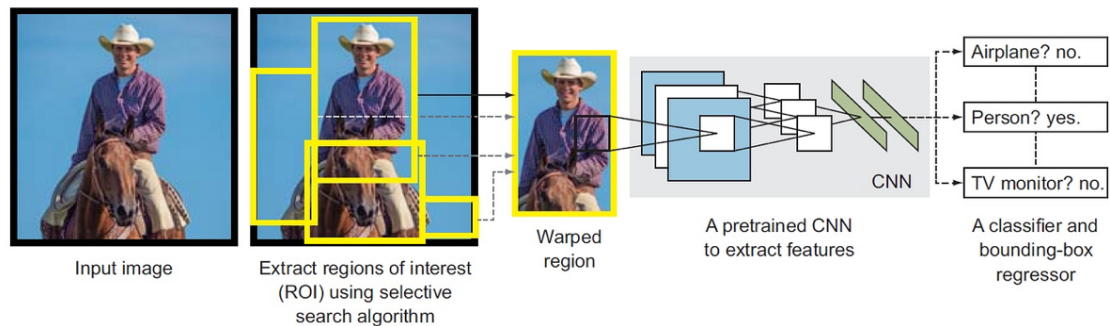
Visual Application

- Visual Understanding
 - Recognition
 - Detection
 - Segmentation
- Multi-Modal Visual Understanding

- Visual Question Answering
- Captioning

Region-Based CNN (RCNN / R-CNN)

R-CNN first proposes RoIs with a search algorithm, then extracts features with CNN, and then classifies those regions based on their features. ROI stands for Region Of Interest, which is literally where we are interested. It refers to places where the object may be in the input image.

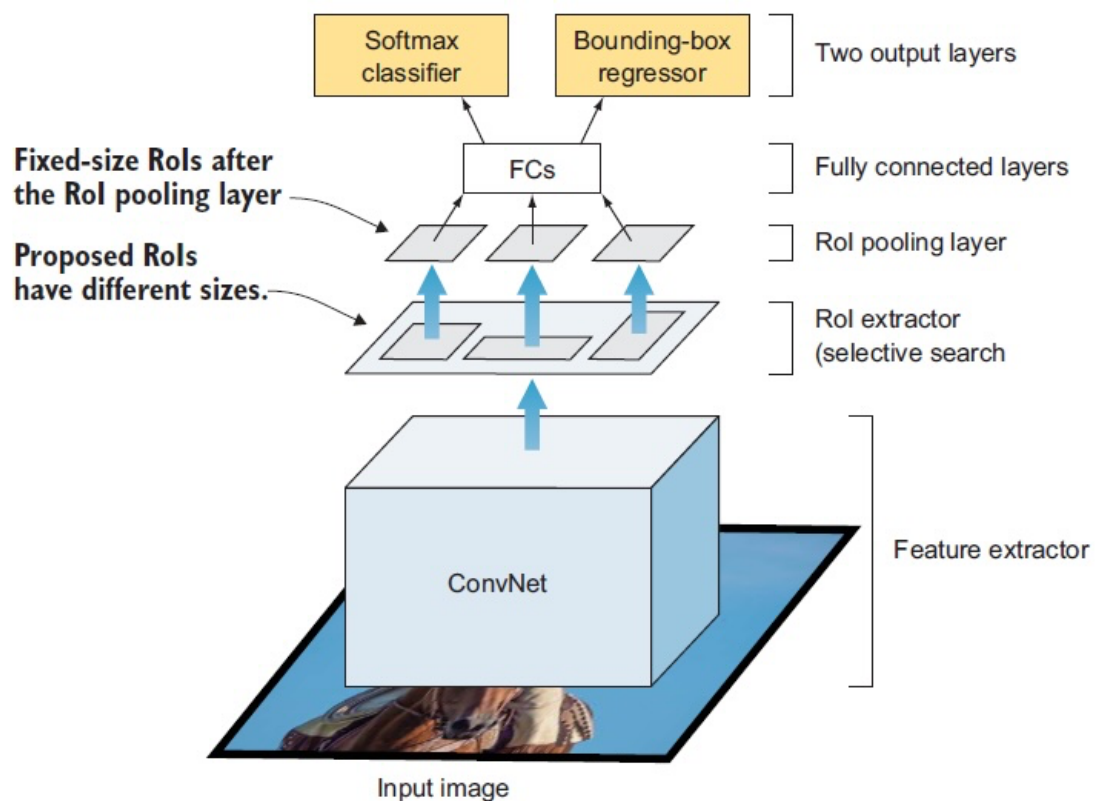


- *Extract region proposals:* These regions have a high probability of containing an object. A selective search algorithm scans the input image to find regions, and proposes them as ROI to be processed by the next modules. The proposed RoIs are then warped to have a fixed size because they usually vary in size, but CNN requires a fixed input image size.
- *Feature extraction module :* A pretrained convolutional network is run to extract features from each ROI.
- *Classification module :* A classifier like SVM is used to classify candidate detections. SVM determines whether the region is an object or not and, if so, what kind of object it is. This is based on the extracted features from the previous step.
- *Localization module :* The location and size of the bounding box that surrounds the object are predicted. The bounding box is represented by identifying four values: the x and y coordinates of the box's origin (x, y), the width, and the height of the box (w, h). Putting this together, the regressor predicts the four real-valued numbers that define the bounding box as the following tuple: (x, y, w, h).

Fast R-CNN

Fast R-CNN produces region proposals based on the last feature map of CNN, not from the original input image like R-CNN. In other words, Fast R-CNN

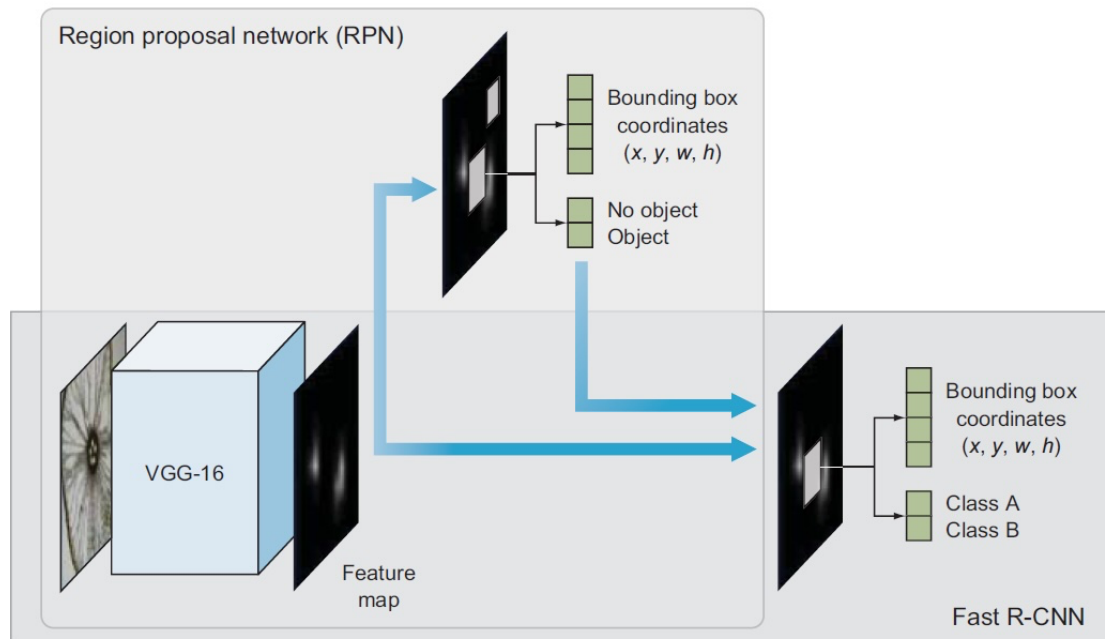
implements CNN first and then generates region proposals. On the contrary, R-CNN produces region proposals first and then implements CNN.



Faster R-CNN

The architecture of Faster R-CNN can be described using two main networks, Region proposal network (RPN) and Fast R-CNN.

The output is then passed into two fully connected layers. One for the object classifier and one for the bounding box coordinate predictions to obtain our final localizations. This architecture achieves an end-to-end trainable, complete object detection pipeline where all of the required components are inside the network

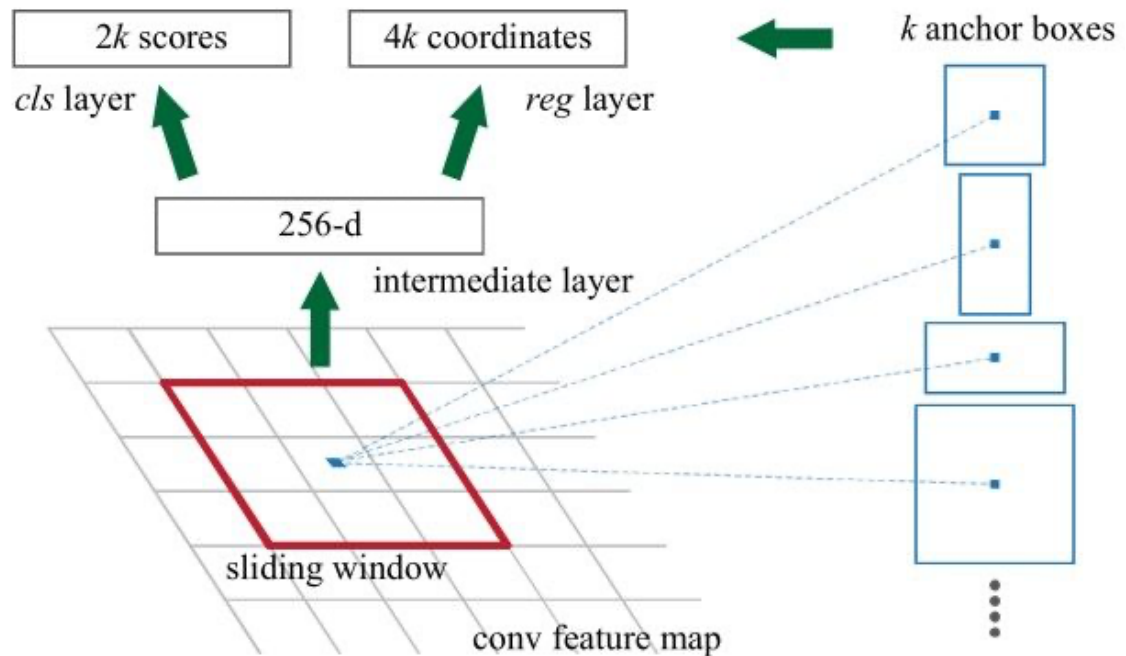


Region Proposal Network (RPN)

RPN has a classifier and a regressor.

- Anchor is the central point of the sliding window.
- Classifier determines the probability of a proposal having the target object.
- Regression regresses the coordinates of the proposals. For any image, scale and aspect-ratio (i.e., width of image/height of image)

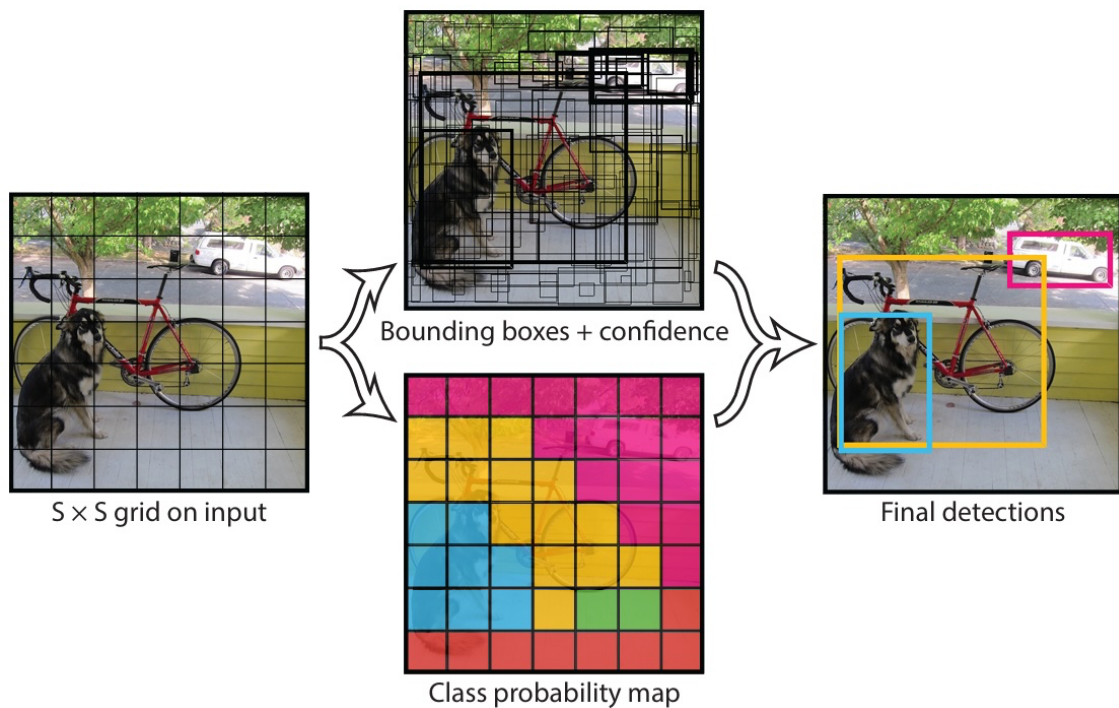
For each pixel (anchor), 3 scale and 3 aspect-ratio are selected ($k=9$). Then, for the whole image, number of anchors is WHK .



You Only Look Once (YOLO)

YOLO [reference](#) is based on the idea of segmenting an image into smaller images. The cell in which the center of an object, for instance, the center of the dog, resides, is the cell responsible for detecting that object.

- Each cell will predict B bounding boxes and a confidence score for each box.
- While each cell may predict any number of bounding boxes and confidence scores for those boxes, it only predicts one class.
 - This is a limitation of the YOLO algorithm itself, and if there are multiple objects of different classes in one grid cell, the algorithm will fail to classify both correctly.
- Each prediction from a grid cell will be of shape $C + B * 5$
 - C is the number of classes and B is the number of predicted bounding boxes
 - B is multiplied by 5 here because it includes $(x, y, w, h, \text{confidence})$ for each box.
 - Because there are $S \times S$ grid cells in each image, the overall prediction of the model is a tensor of shape $S \times S \times (C + B * 5)$.



DEtection TRansformer (DETR)

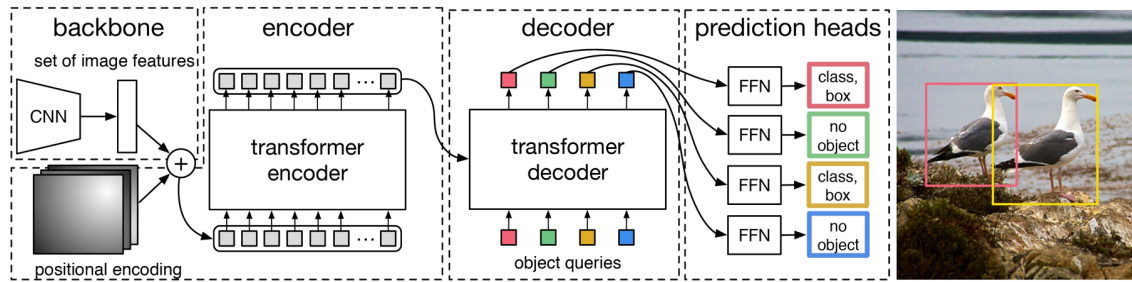
Transformer Encoder-Decoder:

- The encoder takes the features from the backbone and, using self-attention, models the relationships between all parts of the image to create global, context-aware features.
- The decoder takes a fixed number of learned object queries and attends to the encoded image features. Each object query is designed to focus on a potential object in the image and output an embedding for it.

Prediction Feed-Forward Network (FFN): This is a simple feed-forward network that takes each output embedding from the decoder and makes the final prediction: a class label and a bounding box (relative to the entire image, not an anchor).

Bipartite Matching ([Link](#)):

- The Hungarian algorithm is used to find this matching by minimizing the total cost, which combines both the classification error and the bounding box difference.
- After finding the best match, a standard loss (like cross-entropy for classification and a combination of L1 and generalized IoU loss for boxes) is computed only on these matched pairs. This forces the model to learn to make unique predictions for each object.

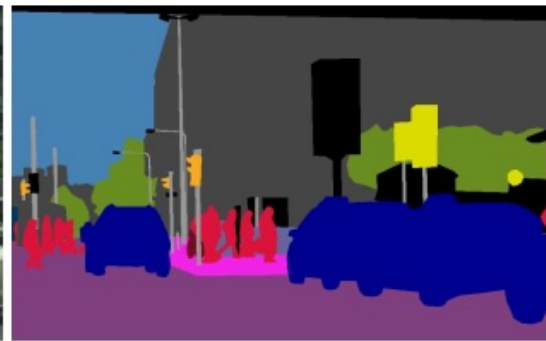


Segmentation

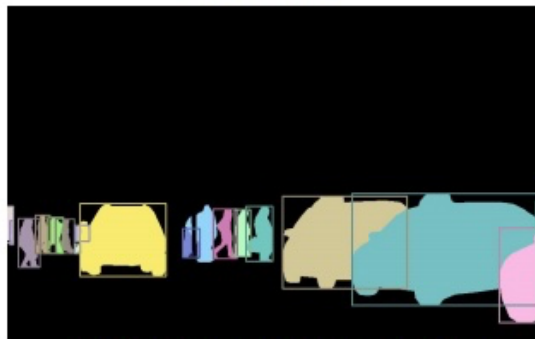
- Semantic, i.e., "What is this pixel?"
- Instance, i.e., "Which object is this pixel?"
- Panoptic, i.e., "What is this pixel AND which object?"



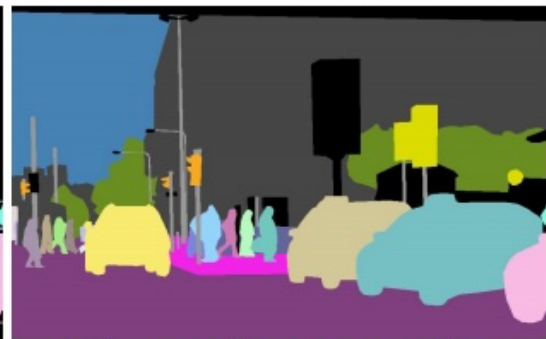
(a) image



(b) semantic segmentation



(c) instance segmentation



(d) panoptic segmentation

Multi-Modal Understanding

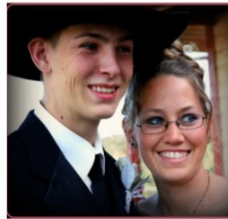
- Vision question answering (VQA): having a machine answer a natural language question about an image.
 - VQA is typically framed as a classification task over a predefined answer vocabulary
 - $P(\text{answer} \mid \text{image}, \text{question}) = \text{softmax}(W * f(\text{image}, \text{question}))$
 - Challenges

- Language Priors: Models can learn to answer based on question patterns without looking at the image. For example, "What color is the banana?" -> "Yellow" (without checking image)
- Compositional Reasoning: Understanding complex questions requiring multiple reasoning steps: "Is the person holding the umbrella because it's raining?"
- Grounding: Ensuring answers are actually grounded in the visual content rather than guessing.

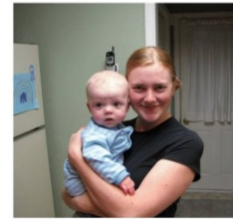
Who is wearing glasses?
man



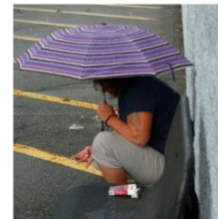
woman



Where is the child sitting?
fridge



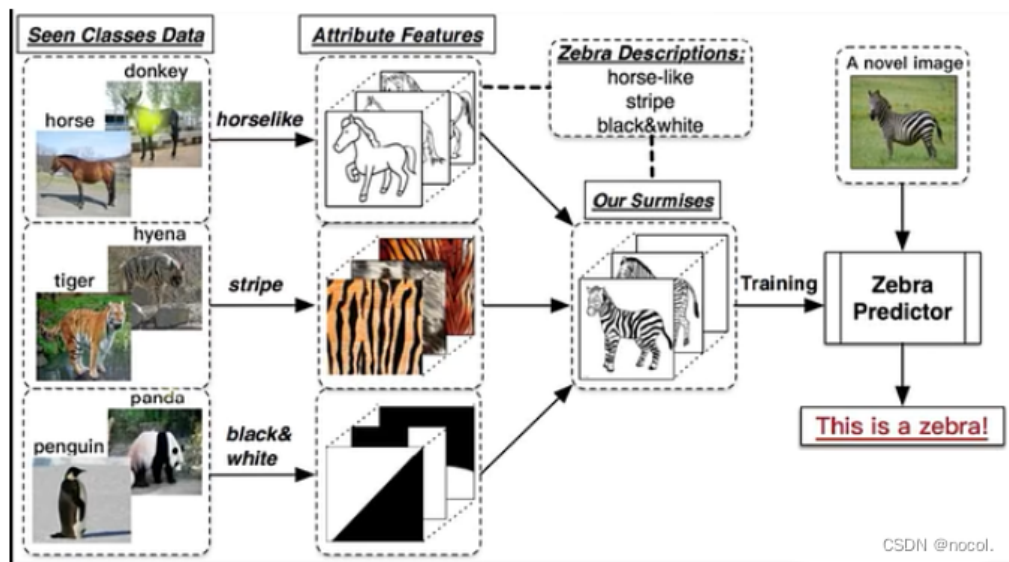
Is the umbrella upside down?
yes



How many children are in the bed?
2



- Zero-shot learning: A machine learning paradigm where a model can recognize and classify objects it has never seen during training.
 - Seen Classes (S): Classes available during training
 - Unseen Classes (U): Classes only encountered during testing
 - Attribute Space: Shared semantic representation
 - Embedding Functions: Map between visual and semantic spaces (A shared space where both seen and unseen classes can be represented using attributes, word vectors, or text descriptions.)



- Open-vocabulary detection: detect and classify objects from any category described in natural language, including categories never seen during training.



(a) Input image



(b) Open-vocabulary 2D detection



(c) Closed-vocabulary 3D detection



(d) Open-vocabulary 3D detection

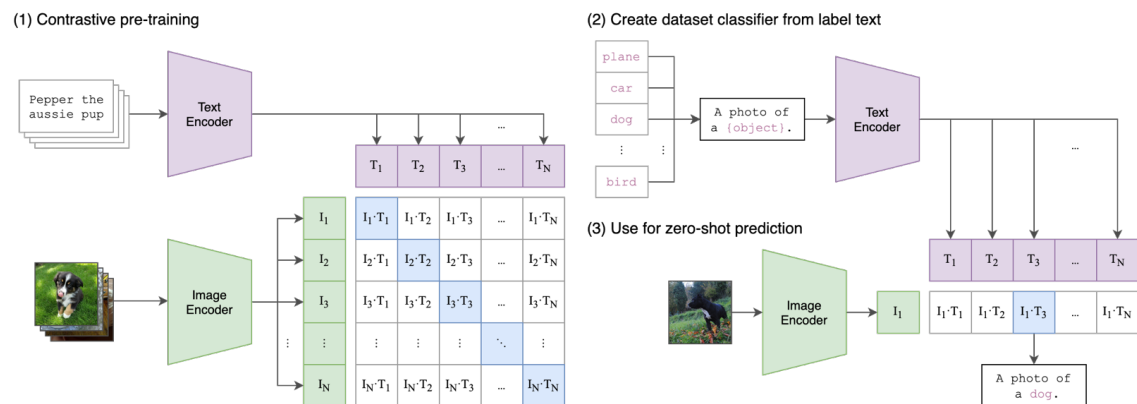
Figure 1. Given (a) a single image, we illustrate examples of (b) open-vocabulary 2D detection, which localizes objects of any category within the 2D image plane, covering both **seen** categories and **novel** categories not seen during training; (c) closed-vocabulary 3D detection, which detects objects from a predefined set of categories in 3D space; and (d) open-vocabulary 3D object detection, which identifies objects of any category in 3D.

Vision Language Models

Contrastive Language-Image Pre-training (CLIP)

CLIP learns visual concepts from natural language supervision by training on millions or billions image-text pairs from the internet (two-tower design)

- Image encoder: Vision Transformer (ViT) or ResNet
- Text encoder: Transformer
- Output: Normalized embeddings in shared space, and compute cosine similarities between all image-text pairs
- Batch processing and Contrastive loss:
 - Maximize similarity for matching pairs
 - Minimize similarity for non-matching pairs in the same batch
 - Symmetric loss: Applied in both directions (image→text and text→image)



```
In [35]: from PIL import Image
import requests

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

# Open image from URL
img_path = "imgs/testimg.jpg" # Image of cats on a couch
raw_image = Image.open(img_path).convert('RGB')
plt.imshow(raw_image, interpolation='nearest')
plt.gca().set_axis_off()
plt.show()

inputs = processor(text=["a photo of a cat", "a photo of a dog"], image=raw_image)

outputs = model(**inputs)
logits_per_image = outputs.logits_per_image # this is the image-text similarity matrix
probs = logits_per_image.softmax(dim=-1)
```



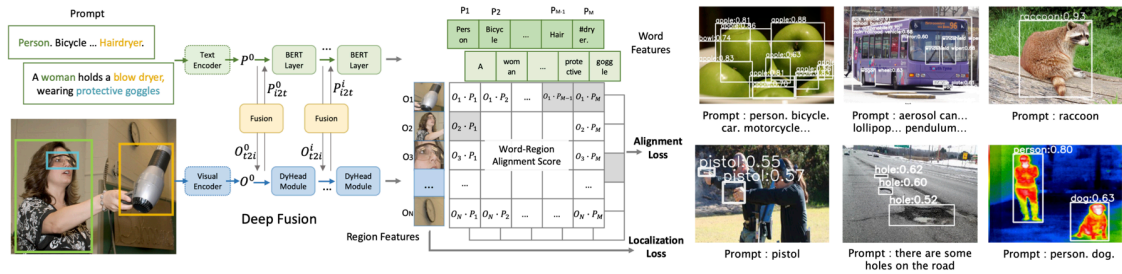
```
In [48]: print(outputs.keys())
print('Text Embedding', outputs.text_embeds.shape)
print('Image Embedding', outputs.image_embeds.shape)

print(probs[0].detach().cpu().numpy())
ind = probs[0].argmax()
print(["a photo of a cat", "a photo of a dog"][ind.item()])

dict_keys(['logits_per_image', 'logits_per_text', 'text_embeds', 'image_embeds', 'text_model_output', 'vision_model_output'])
Text Embedding torch.Size([2, 512])
Image Embedding torch.Size([1, 512])
[0.994858  0.00514195]
a photo of a cat
```

Grounded Language-Image Pre-training (GLIP)

Grounded Language-Image Pre-training (GLIP) is a model that unifies object detection and language understanding. It learns by aligning regions in an image with words or phrases in a text caption, allowing it to perform tasks like zero-shot object detection based on any text prompt



Vision Question Answering

```
In [49]: # Define Model and Device (use "cuda" if GPU is available, otherwise "
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
MODEL_ID = "Salesforce/blip-vqa-base"

# Example Image URL and Question
question_text = "how many cats are in the picture?"

vqa_pipeline = pipeline(
    task="visual-question-answering",
    model=MODEL_ID,
    device=0 if DEVICE == "cuda" else -1 # Use GPU 0 or CPU -1
)

# Execute inference
# Output is a list of dictionaries [{'score':..., 'answer': '...'}]
pipeline_output = vqa_pipeline(image=raw_image, question=question_text)
print('Question:', question_text)
print(f"Final Answer: {pipeline_output[0]['answer']}")
```

Device set to use cpu

Question: how many cats are in the picture?

Final Answer: 2

```
In [32]: MODEL_ID = "runwayml/stable-diffusion-v1-5"

# Load the pipeline, moving components (Text Encoder, U-Net, VAE) to t
pipeline = AutoPipelineForText2Image.from_pretrained(
    MODEL_ID,
    torch_dtype=torch.float16 if DEVICE == "cuda" else torch.float32,
).to(DEVICE)

# Define the text prompt
prompt = "Astronaut riding a neon space horse, detailed digital art, 8

# Define Generation Parameters for reproducibility and quality control
SEED = 42
GUIDANCE_SCALE = 7.5
INFERENCE_STEPS = 30

# Initialize the Generator for reproducible results
```

```
generator = torch.Generator(device=DEVICE).manual_seed(SEED)

# Execute Inference
print(f"Starting image generation for prompt: '{prompt}'")
print(f"Using Seed: {SEED}, Guidance Scale: {GUIDANCE_SCALE}, Steps: {
    STEPS}")

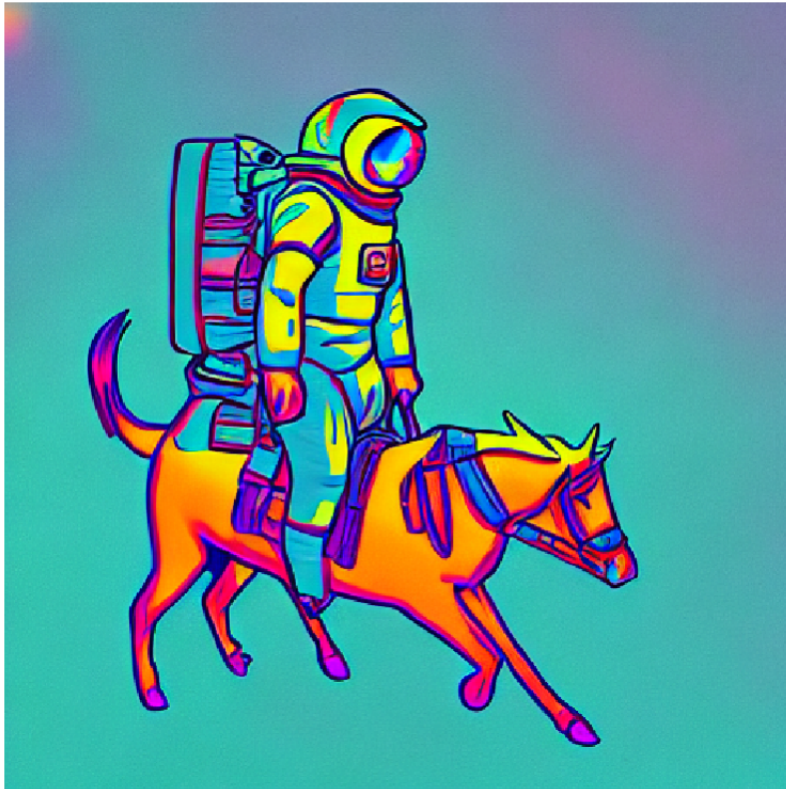
# The pipeline executes the full LDM flow: Text Encoding -> Latent Inference
# Iterative U-Net Denoising -> VAE Decoding [24]
output = pipeline(
    prompt=prompt,
    num_inference_steps=INFERENC
E_STEPS,
    guidance_scale=GUIDANCE_SCALE,
    generator=generator,
)

# The output is a list containing the PIL Image object
generated_image = output.images[0]
plt.imshow(generated_image, interpolation='nearest')
plt.gca().set_axis_off()
plt.show()
```

```

Loading pipeline components...: 0%|
| 0/7 [00:00<?, ?it/s]huggingface/tokenizers: The current process just
got forked, after parallelism has already been used. Disabling parallel
ism to avoid deadlocks...
To disable this warning, you can either:
    - Avoid using `tokenizers` before the fork if possible
    - Explicitly set the environment variable TOKENIZERS_PARALLELIS
M=(true | false)
`torch_dtype` is deprecated! Use `dtype` instead!
Loading pipeline components...: 100%|
| 7/7 [00:00<00:00, 28.85it/s]
Starting image generation for prompt: 'Astronaut riding a neon space ho
rse, detailed digital art, 8k'
Using Seed: 42, Guidance Scale: 7.5, Steps: 30
100%|
| 30/30 [01:30<00:00, 3.01s/it]

```

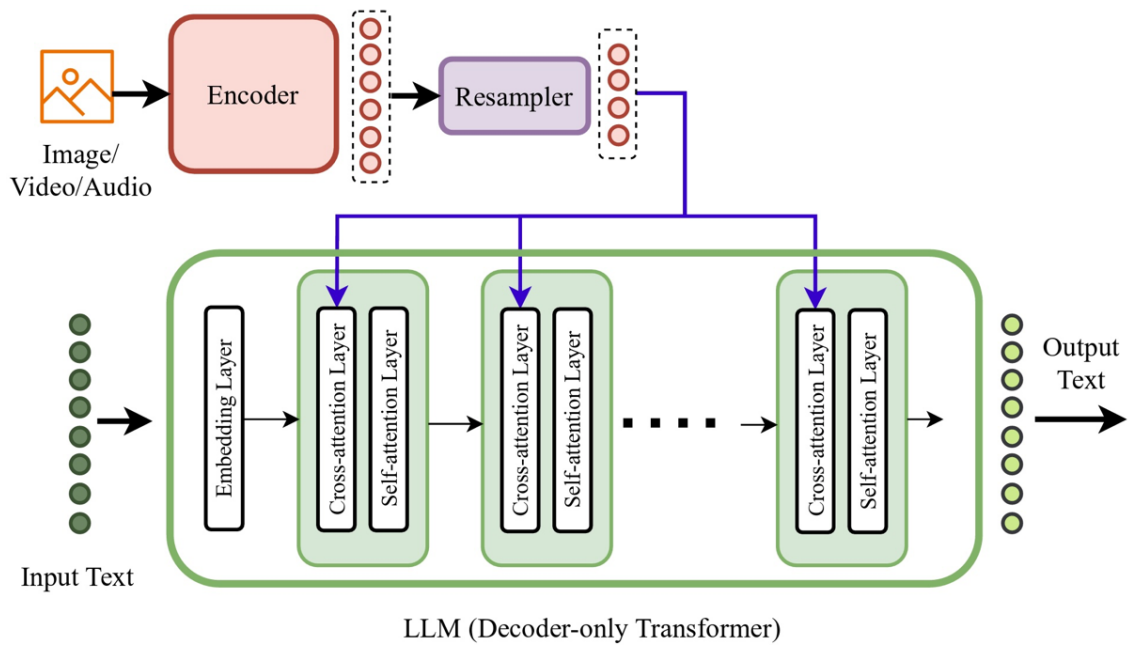



Bias and Ethics Issues

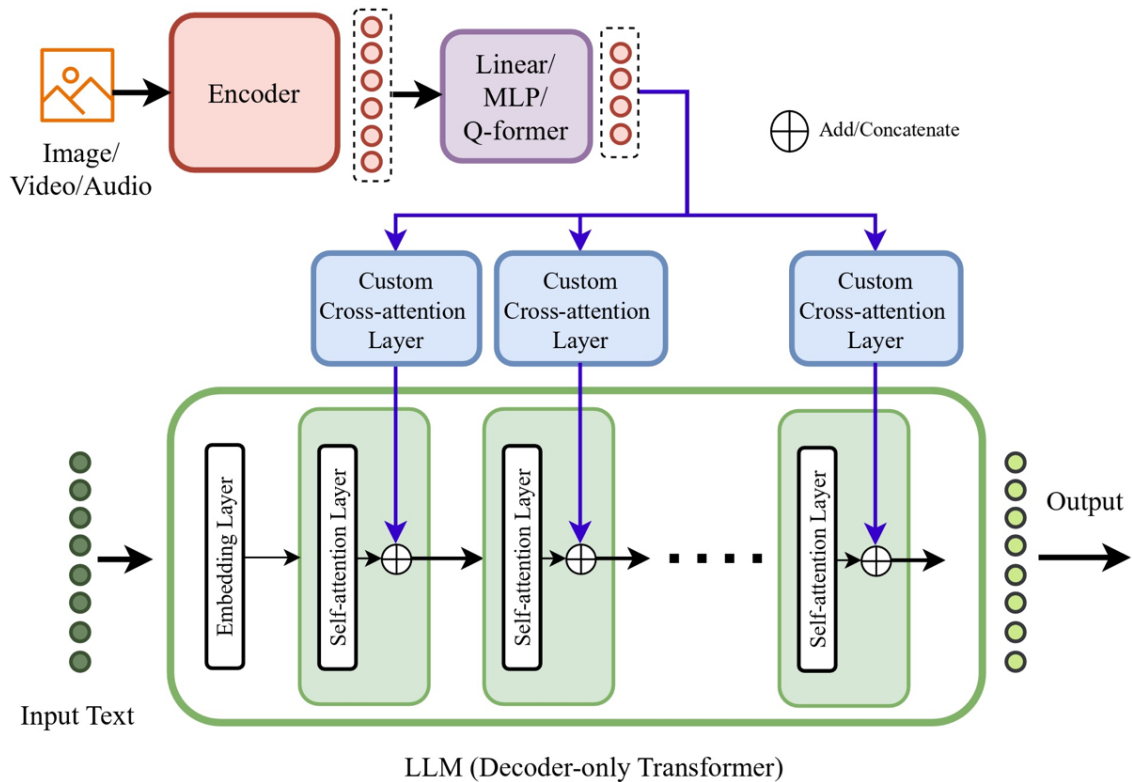
- Bias Issues: Stereotypes, Representational Bias, Algorithmic Discrimination, Toxic and Harmful Content
- Ethics Issues: Privacy Concerns, Intellectual Property and Copyright, Job Displacement, Environmental Impact
- We need to carefully think more ...

Multi-Modal Large Language Models (MLLM)

Standard Cross-Attention based Deep Fusion: The input modalities are deeply fused into the internal layers of the LLM using standard cross-attention layer. The cross-attention can be added either before or after the self-attention layer



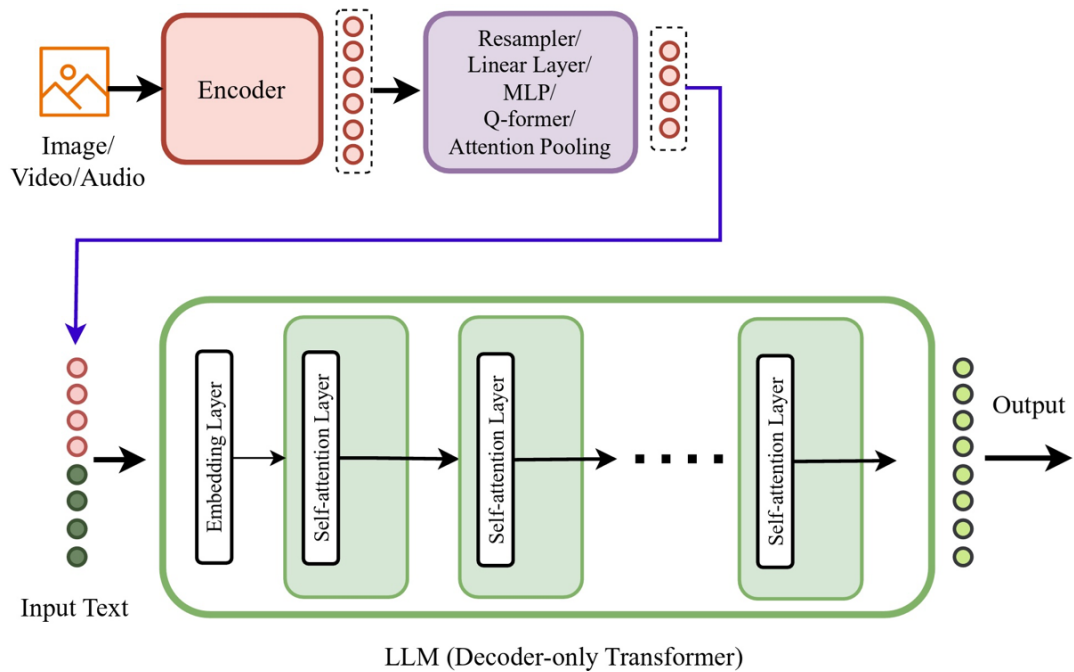
Custom Layer based Deep Fusion (CLDF): The input modalities are deeply fused into the internal layers of the LLM using custom-designed layers. Custom cross-attention layers or other custom layers are used for modality fusion.



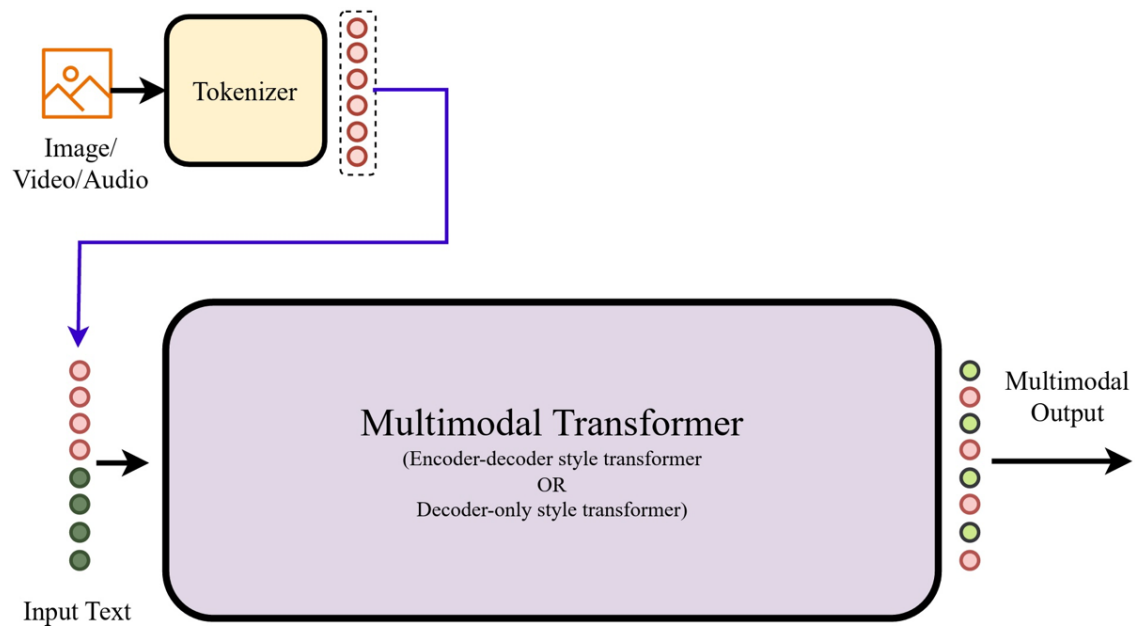
Non-Tokenized Early Fusion (NTEF): The (non-tokenized) input modalities are directly fed to the model at its input, rather than to its internal layers, resulting in early fusion. Different types of modules are used to connect modality encoder outputs to the LLM (model) like

- a LinearLayer/MLP,

- Q-former and a Linear-Layer/MLP
- Perceiver resampler
- Custom learnable layers



TokenizedEarlyFusion(TEF): The tokenized input modalities are directly fed into the model at its input. Either a decoder-only transformer or an encoder-decoder style transformer is used as the multimodal transformer in this architecture.



VLMs: Applications and Future

- Applications:

- multi-modal Chatbot
- GUI-agent, embodied AI (robots), spatial intelligence etc
- Main research questions:
 - How can VLMs achieve better compositional reasoning and reduce object hallucination?
 - How to efficiently scale VLMs while maintaining performance on specialized domains?
 - How can VLMs effectively handle video, i.e., spatial-temporal understanding?
 - How can VLM facilitate robot learning and embodied AI design.

Reference

- <https://github.com/m-peker/visionbridge-vm>
- <https://huggingface.co/blog/vlms-2025>
- <https://medium.com/@msapeker/building-vision-language-models-from-scratch-a-practitioners-guide-b092a29105cf>